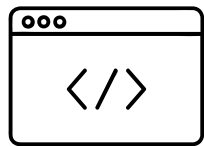
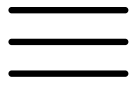


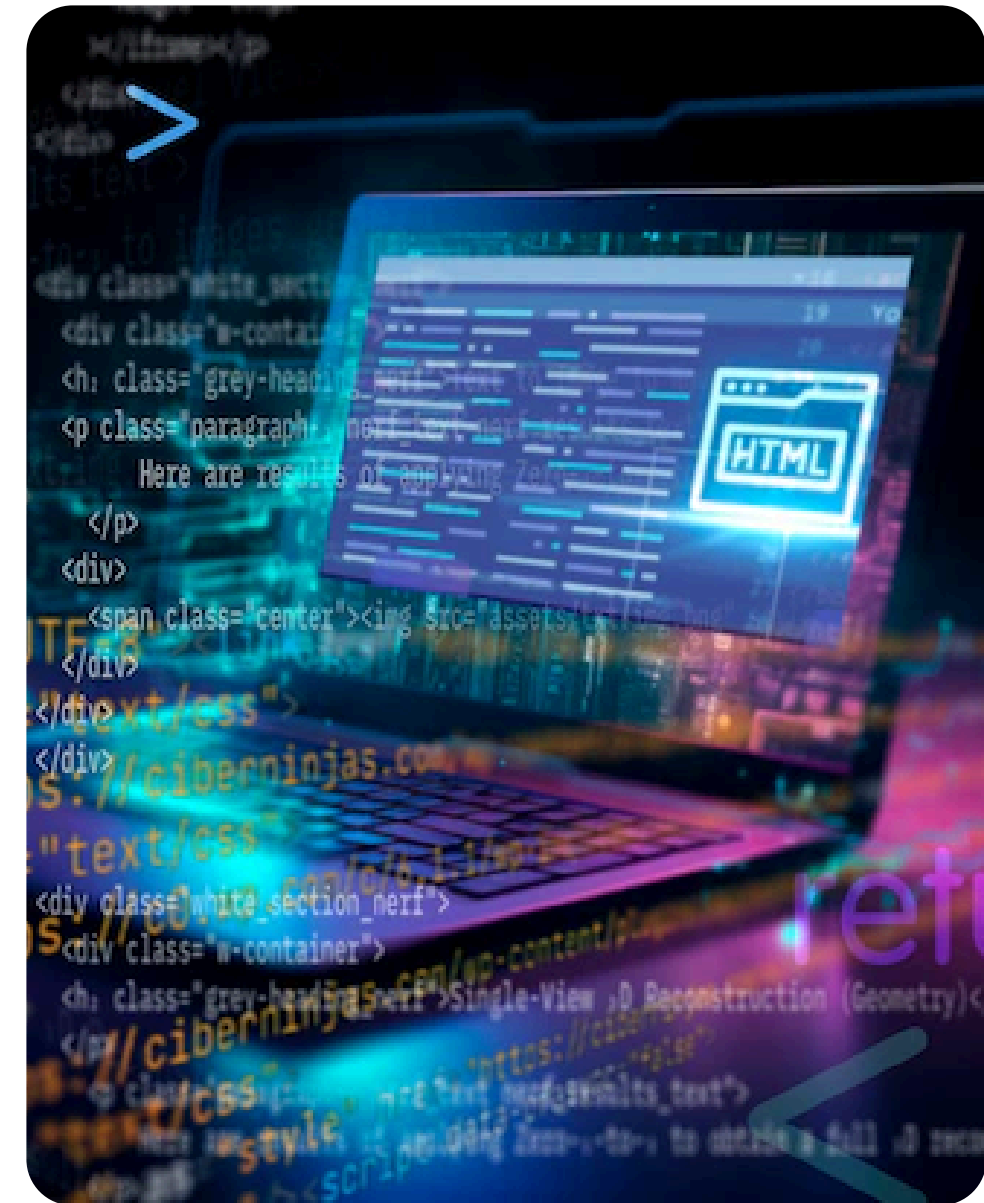
Apuntadores

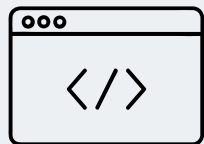
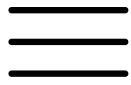
Programación Avanzada



Agenda:

1. Concepto de apuntador
2. Paso por valor y referencia
3. Apuntadores y variables
4. Apuntadores y arreglos
 - a. Aritmética de apuntadores
 - b. Arreglos n-dimensionales
5. Apuntadores y estructuras





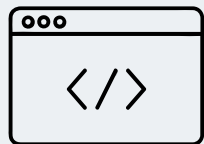
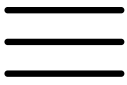
Apuntadores

Tipos de datos específicos para denotar posiciones en memoria

```
#include <iostream>

using namespace std;

int main() {
    int a = 10;
    int* ptr = &a;
    cout << *ptr << endl; // 10
    return 0;
}
```



Paso por referencia

Paso por valor: El sistema crea una copia de los valores de las variables en una variable de contexto dentro de la función.

Paso por referencia: Enviar la misma posición en memoria de la variable del contexto para poder cambiar su valor.

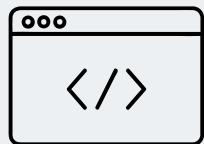
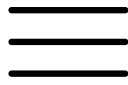
```
#include <iostream>

using namespace std;

void func(int a) { a++; }

void func2(int* a) { *a += 1; }

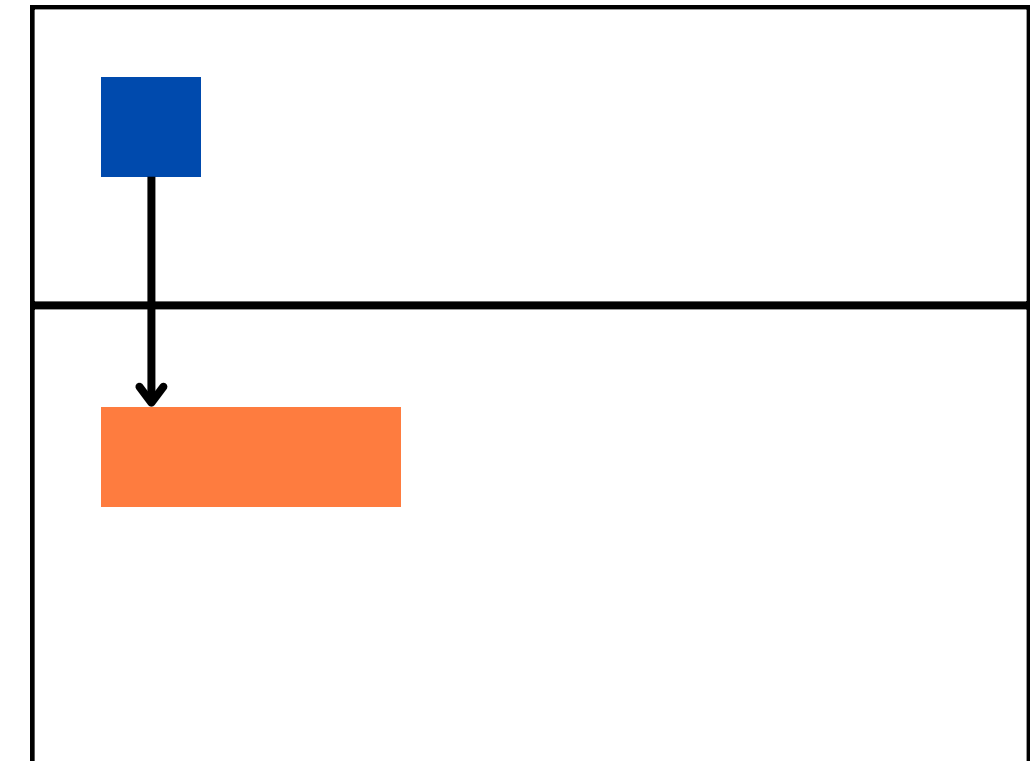
int main() {
    int a = 10;
    func(a);
    int* ptr = &a;
    func2(ptr);
    cout << a << endl; // 11
    return 0;
}
```

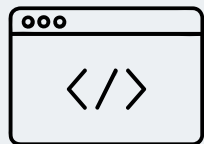
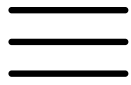


Stack vs Heap

Stack: Memoria del sistema para definiciones (variables, constantes, funciones) gestionada de manera automática.

Heap: Memoria dinámica, de mayor capacidad, gestionada de manera explícita o manual.





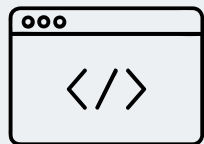
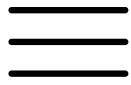
Variables

Se puede definir un apuntador y directamente asignar una posición aleatoria de memoria.

```
#include <iostream>

using namespace std;

int main() {
    int* ptr = new int(10);
    cout << *ptr << endl; // 10
    return 0;
}
```



Arreglos

El compilador de C++ sistema asigna los arreglos de forma eficiente por medio de apuntadores, pero estos pueden ser definidos directamente en el Heap para almacenar una mayor informacion.

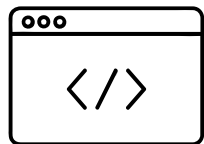
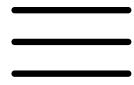
```
#include <iostream>

using namespace std;

int main() {
    // Definicion normal
    int arr[] = { 1, 2, 3, 4, 5};
    cout << arr << endl; // 0xae1bffa40

    // Definicion por puntero
    int* arr2 = new int[5];

    return 0;
}
```



Aritmética de Apuntadores

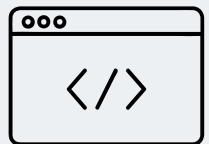
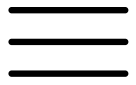
Programación Avanzada

Aritmética de Apuntadores

Se hace referencia a las posiciones de los arreglos a traves de un desplazamiento del apuntador sobre los bytes de la estructura de datos

```
int arreglo[5];  
arreglo[1] = 123;  
cout << arreglo[1] << endl; // 123
```

```
int* arreglo = new int[5];  
*(arreglo + 1) = 123;  
cout << *(arreglo + 1) << endl; // 123
```



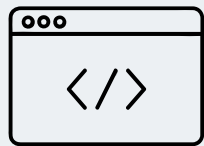
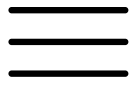
Arreglos n-dimensionales

Se almacenan como arreglos cuyo contenido son apuntadores, de los cuales cada uno de ellos puede ser asignado como un arreglo en si mismo.

```
#include <iostream>

using namespace std;

int main()
{
    int n = 10, m = 5;
    int** arreglo = new int*[n];
    for(int i = 0; i < n; i++){
        *(arreglo + i) = new int[m];
    }
    return 0;
}
```



Estructuras

Las estructuras pueden ser almacenada de manera completa en el Heap utilizando su constructor.

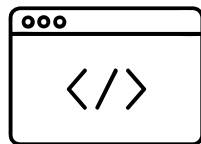
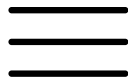
- El operador de flecha ($\rightarrow$) se utiliza para hacer alusión a la estructura directamente desde su posición en memoria.

```
#include <iostream>

using namespace std;

struct Persona {
    string nombre;
    int edad;
    Persona(string nombre, int edad) {
};

int main() {
    Persona* p = new Persona("Jimmy", 26);
    cout << p->nombre << endl; // Jimmy
    return 0;
}
```



Actividad:

Ordenamiento: Construir un sistema el cual, a partir de un arreglo de tamaño 10 creado con numero aleatorios entre el 1 y el 100, cree un arreglo auxiliar con sus posiciones en memoria y realice un ordenamiento sobre este sin alterar el arreglo original.

Es obligatorio el uso de aritmética de apuntadores.

```
Original: | 83 || 75 || 85 || 12 || 90 |  
Ordenado: | 12 || 75 || 83 || 85 || 90 |
```



Gracias

Programación Avanzada